

UNIVERSIDAD ARGENTINA DE LA EMPRESA

Facultad de Ingeniería y Ciencias Exactas (FAIN)

Ingeniería Electrónica

UADE

SISTEMAS EMBEBIDOS

Trabajo Práctico Integrador

**Interfaz física de operación segura para un robot Unitree: Deadman switch
con detección de presencia sobre Blue Pill (STM32F103)**

Integrantes del grupo

Apellido y Nombre: BOLANCA, Santiago — LU: 1176307

Apellido y Nombre: MARIANI, Máximo — LU: 1151848

Apellido y Nombre: MAYER, Emiliano — LU: 1187028

Apellido y Nombre: GRAPUNSKY, Gastón — LU: 1179114

Docente: Prof. Adj. Dr. Ing. Leonardo J. Amet

Cuatrimestre: 1.º Cuatrimestre 2026

Fecha de entrega: 15 / 07 / 2026

Índice

Índice	2
1. Introducción.....	4
1.1. Descripción general del proyecto	4
1.2. Motivación	5
1.3. Objetivos.....	5
2. Marco teórico	6
2.1. El microcontrolador STM32F103C8T6 (Blue Pill).....	6
2.2. Base de firmware: libopencm3 + FreeRTOS	6
2.3. El protocolo serie (la base del TP5).....	7
2.4. Periféricos empleados.....	9
2.5. Sensores de presencia: PIR e inclinación	10
3. Diseño del sistema	12
3.1. Diseño de hardware.....	12
3.2. Diseño de software	15
4. Implementación y montaje	18
4.1. Montaje en protoboard	18
4.2. Fabricación de la PCB	18
4.3. Ensamblaje de componentes	19
5. Pruebas y validación.....	22
5.1. Configuración del entorno de pruebas	22
5.2. Ensayos experimentales y resultados.....	22
6. Especificaciones técnicas	27
7. Procedimientos de calibración y ajustes	28
7.1. Ajustes de hardware	28
7.2. Ajustes en el código	28
8. Registro anecdótico y seguimiento de fallas	29
8.1. El LED RGB que casi no encendía (y el azul que no encendía nunca)	29
8.2. El pin PB4 que no era un GPIO	29
8.3. El botón fantasma.....	29
8.4. El tilt que bloqueaba todo	30
8.5. El Makefile que flasheaba el firmware equivocado	30
9. Parte comercial	31
9.1. Costos y presupuesto.....	31

10. Conclusiones	32
11. Posibilidades de mejora	33
12. Manual de usuario	34
12.1. Normas de seguridad, precauciones y recomendaciones	34
12.2. Instrucciones de uso	34
13. Bibliografía y recursos	36
14. Anexos	37
Anexo A: repositorio del proyecto	37
Anexo B: compilación y flasheo	37

1. Introducción

1.1. Descripción general del proyecto

En este trabajo integrador construimos un **mando de operador con lógica de hombre muerto** (deadman switch) para un robot cuadrúpedo Unitree. El corazón del sistema es una placa **Blue Pill (STM32F103C8T6)** que no controla al robot en forma directa: se comunica por **UART** con una computadora que corre un **bridge ROS 2** provisto por la cátedra, usando un protocolo serie propio con tramas, longitud declarada y checksum. El bridge traduce esas tramas en tópicos y comandos que el robot entiende.

La regla de operación es simple de enunciar y estricta de cumplir: **el robot solo se mueve mientras el operador está presente**. Para nosotros, “presente” no es solamente un botón apretado: sumamos una capa de **detección de presencia** con un sensor PIR (que verifica que haya una persona frente al puesto) y un sensor de inclinación (que verifica que el mando esté derecho y no, por ejemplo, tirado en el piso). Si cualquiera de las tres condiciones falla, la placa ordena la parada inmediata del robot y lo señala con luz y sonido.

Mientras el operador está presente, un potenciómetro define la **velocidad de avance** (0–100 %), un LED RGB indica el estado con el brillo del verde proporcional a la velocidad, y un buzzer emite un tono cuya frecuencia también acompaña a la velocidad. Todo el circuito se montó primero en protoboard y después en una **PCB propia de una sola capa**, con forma de joystick de consola.



Figura 1. Robot cuadrúpedo Unitree Go2, la plataforma a la que se conecta el sistema a través del bridge ROS 2 de la cátedra.

1.2. Motivación

El deadman switch es un mecanismo de seguridad real y muy difundido: lo llevan los trenes, las máquinas herramienta y los equipos de gran porte. La idea de fondo es que la máquina requiera una **acción sostenida y consciente** del operador para funcionar: si el operador se desmaya, se distrae o suelta el control, la máquina se detiene sola. Nos pareció el mejor marco posible para un robot que camina: es un caso donde el firmware embebido no es un ejercicio de laboratorio sino la capa que decide si un equipo físico se mueve o no.

También nos motivaba cerrar la materia integrando de verdad todo lo que fuimos viendo: el protocolo serie y el parser que escribimos en el TP5, los periféricos (ADC, PWM, EXTI, timers) de los TP anteriores, FreeRTOS del TP4, y sumarle algo que nos quedaba pendiente como grupo: **diseñar y fabricar nuestra propia PCB** en KiCad.

1.3. Objetivos

- ▶ Implementar el **protocolo serie completo** (tramas, checksum XOR, parser incremental con FSM y resincronización) sobre UART, reutilizando la base validada del TP5.
- ▶ Integrar el **núcleo de periféricos** del enunciado: ADC (potenciómetro), PWM (LED y buzzer), LED, buzzer y botón por EXTI con anti-rebote.
- ▶ Implementar la **variante 2 (deadman switch)** completa: velocidad por potenciómetro, heartbeat, parada al soltar y reenvío del evento hasta recibir el ACK del bridge.
- ▶ Sumar una capa original de **presencia del operador** con sensores PIR y de inclinación, sin modificar el contrato del protocolo.
- ▶ Diseñar y fabricar una **PCB de una sola capa** en KiCad, con componentes THT y revelado casero con percloruro.
- ▶ Producir **evidencia concreta** de funcionamiento: tramas en el monitor serie, tópicos del bridge, comportamiento de los periféricos y reacción del robot.

2. Marco teórico

2.1. El microcontrolador STM32F103C8T6 (Blue Pill)

La Blue Pill es una placa de desarrollo económica construida alrededor del **STM32F103C8T6** de ST: un microcontrolador de 32 bits con núcleo **ARM Cortex-M3** que corre hasta **72 MHz**, con **64 KB de flash** y **20 KB de SRAM**. Para este trabajo, lo que más nos importa es su dotación de periféricos: conversor A/D de 12 bits, varios timers de propósito general con salidas de comparación (PWM), USART, y un controlador de interrupciones externas (EXTI) que permite reaccionar a flancos en cualquier pin.

La placa expone dos filas de 20 pines con paso de 2,54 mm, se alimenta por USB (5 V) y genera internamente los 3,3 V de la lógica. Ese detalle —que **el micro trabaja a 3,3 V**— condiciona todo el diseño eléctrico: nada de 5 V directo a un pin de señal.

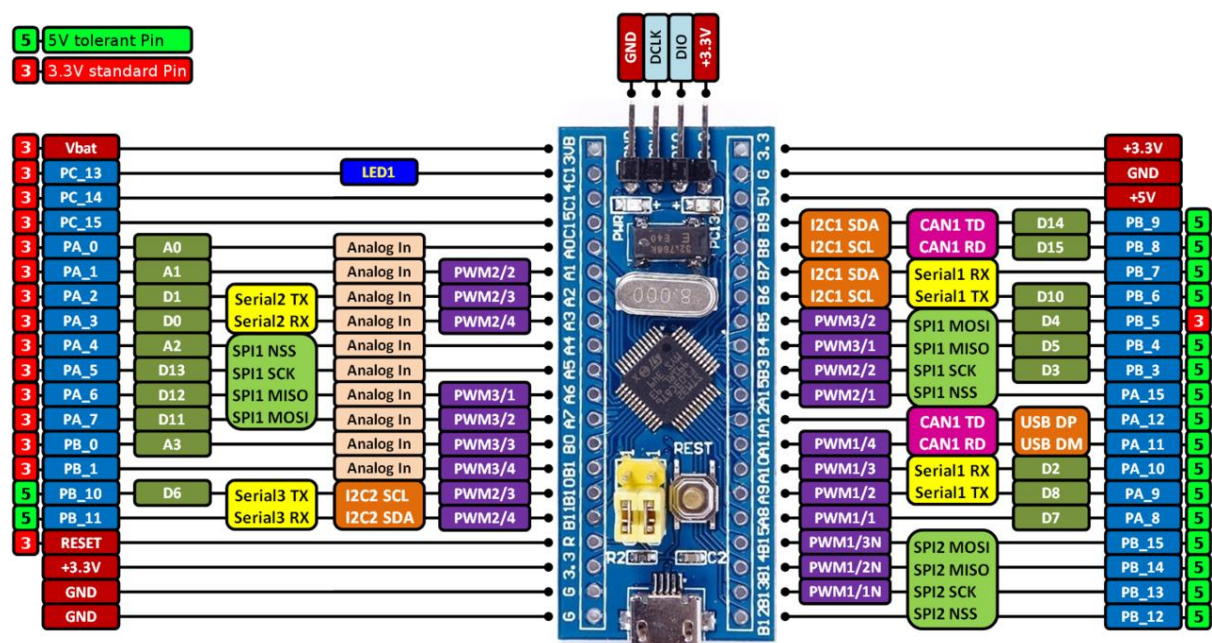


Figura 2. Distribución de pines de la Blue Pill. Usamos los puertos A y B para todos los periféricos del trabajo.

2.2. Base de firmware: libopencm3 + FreeRTOS

El firmware se apoya en el esqueleto de la cátedra, que combina dos piezas. La primera es **libopencm3**, una biblioteca open source que envuelve los registros del STM32 en funciones con nombre (por ejemplo `timer_set_prescaler()` en lugar de escribir a mano el registro PSC). No esconde el hardware: uno sigue decidiendo cada valor, pero el código queda legible.

La segunda es **FreeRTOS**, el sistema operativo de tiempo real que vimos en la cursada. Nos da **tareas** (funciones que corren “en paralelo” con prioridades) y **colas** (buffers seguros para pasar datos entre tareas y desde interrupciones). La gracia de esta arquitectura es que cada problema queda aislado en su tarea: una tarea recibe bytes, otra parsea, otra decide, otra

transmite. Si mañana cambia el medio físico (por ejemplo, de UART a otra interfaz), el resto del sistema ni se entera.

El arranque del micro (startup, tabla de vectores y script del linker) ya lo estudiamos en detalle en el TP1; acá lo hereda el esqueleto: el archivo `linker.ld` define dónde va cada sección en flash y RAM, y el startup copia los datos inicializados, limpia el BSS y salta a `main()`.

2.3. El protocolo serie (la base del TP5)

Toda la comunicación entre la Blue Pill y el bridge usa un formato de trama fijo que diseñó la cátedra y que nosotros implementamos y validamos en el TP5:

```
@LL:TTT:PAYLOAD:CC\n
```

El `@` marca el inicio; `LL` es la longitud del cuerpo `TTT:PAYLOAD` en hexadecimal de dos dígitos; `TTT` es el tipo de mensaje (tres letras mayúsculas); `PAYLOAD` es el contenido en ASCII imprimible; `CC` es el checksum, y el salto de línea cierra la trama.

Tabla 1. Tipos de mensaje del protocolo y su uso en este trabajo.

Tipo	Dirección	Uso en la variante deadman
EVT	Pill → Bridge	Eventos discretos: <code>deadman_ok</code> (operador presente), <code>deadman</code> (parada)
DAT	Pill → Bridge	Datos periódicos: <code>spd=NN</code> (velocidad 0–100), <code>adc=NNNN</code> (crudo)
STS	Pill → Bridge	Estado: <code>ALIVE</code> (heartbeat), <code>EMERGENCY</code> (parada por sensor)
ACK	Bridge → Pill	Confirmación de recepción (ok tras cada <code>EVT:deadman</code>)
ERR	Pill → Bridge	Errores (fuera de rango, timeout)
CMD	Bridge → Pill	Comandos hacia la Pill (no se usa en esta variante)

2.3.1. El checksum XOR

El checksum se calcula aplicando XOR byte a byte sobre la cadena `LL:TTT:PAYLOAD`. Si un byte cualquiera se corrompe en el camino, el resultado cambia y el receptor descarta la trama. Formalmente, si la cadena tiene N caracteres $c_1 \dots c_N$:

$$CC = c_1 \oplus c_2 \oplus \dots \oplus c_N \quad (1)$$

La implementación es deliberadamente simple (es la misma función del TP5, sin cambios):

```
firmware/protocol/protocol.c
```

```
uint8_t protocol_compute_checksum(const char *data, size_t length)
{
    uint8_t checksum = 0U;
    for (size_t i = 0U; i < length; i++) {
        checksum ^= (uint8_t) data[i];
    }
}
```

```
    return checksum;
}
```

Como ejemplo completo: para el payload `adc=2048` el cuerpo es `DAT:adc=2048` (12 caracteres → `LL = 0C`), el XOR de `0C:DAT:adc=2048` da `0x77`, y la trama final queda `@0C:DAT:adc=2048:77\n`.

QUÉ GARANTIZA (Y QUÉ NO) EL XOR

El XOR detecta cualquier error de un solo bit y el cambio de un byte aislado. No detecta el intercambio de dos bytes (es conmutativo) ni errores que se compensan entre sí. Para más robustez sin cambiar la trama, el paso siguiente natural sería un CRC-8, que ocupa los mismos dos dígitos hexadecimales.

2.3.2. El parser incremental como máquina de estados

Del lado receptor, los bytes llegan como llegan: una trama puede venir partida entre interrupciones, pegada a la siguiente o precedida de ruido. Por eso el parser **no espera líneas completas**: consume un byte por vez y recuerda en qué parte de la trama está parado. Cada estado responde a la pregunta “¿qué byte espero ahora?”.

Tabla 2. Los nueve estados del parser (implementado y validado en el TP5).

Estado	Espera	Acción
WAIT_START	@	Ignora todo hasta el inicio de trama
READ_LEN_HI / LO	dígito hex	Arma la longitud LL
EXPECT_LEN_SEPARATOR	:	Confirma el separador
READ_BODY	LL bytes	Acumula TTT:PAYLOAD contando bytes (no busca delimitadores)
EXPECT_CHECK_SEPARATOR	:	Confirma el separador del checksum
READ_CHECK_HI / LO	dígito hex	Arma el CC recibido
EXPECT_END	\n	Recalcula el checksum, valida tipo y payload, y entrega el mensaje

La pieza más fina es la **resincronización**: ante cualquier error el parser descarta la trama parcial y vuelve a `WAIT_START`; pero si el byte que causó el error es una `@`, la reutiliza como inicio de una trama nueva. Ese detalle es la diferencia entre perder o recuperar una trama válida cuando hay ruido en la línea: en la secuencia `@0@08:CMD:ping:52` el fragmento `@0` se rompe al llegar la segunda arroba, pero esa misma arroba arranca la trama buena y el `ping` se entrega igual.

POR QUÉ LA BASE DEL TP5 SE REUTILIZÓ SIN TOCAR

El protocolo separa transporte de significado: el parser garantiza que el mensaje llegue completo, tipado y sin errores, y no le importa qué dice el contenido. Gracias a esa frontera, el mismo código que en el TP5 prendía un LED hoy gobierna la parada de emergencia de un robot: solo cambió la capa de aplicación.

En el TP5 verificamos esta lógica con tests de host (38 casos en total entre protocolo y parser, corriendo en la PC sin hardware) y después contra la placa real con un script de Python. Esa validación previa es la que nos permitió arrancar el integrador con la comunicación resuelta.

```
maximomariani@PC-DANIEL:~/BLUEPILL$ python3 tp5-embebidos-mariani/codigo/serial_test/tp5_serial_test.py /dev/ttyUSB0
=== TP5: prueba del protocolo contra la Blue Pill (/dev/ttyUSB0, 115200 8N1) ===

--- Comandos normales (esperan ACK) ---
> ping                TX @08:CMD:ping:52\n
                      RX @0A:ACK:pong=1:22\n

> led=on              TX @0A:CMD:led=on:6A\n
                      RX @0A:ACK:cmd=ok:6B\n

> led=off             TX @0B:CMD:led=off:07\n
                      RX @0A:ACK:cmd=ok:6B\n

> led=toggle          TX @0E:CMD:led=toggle:7D\n
                      RX @0A:ACK:cmd=ok:6B\n

--- Estado (espera STS con contadores) ---
> status?             TX @0B:CMD:status?:13\n
                      RX @2B:STS:rx=5,ae=0,irq=126,pb=126,pm=5,pe=2,qd=0:40\n

--- Errores (esperan ERR) ---
> comando desconocido TX @0E:CMD:frobinate:32\n
                      RX @14:ERR:code=unknown_cmd:2D\n

> tipo inesperado     TX @08:ACK:hola:4B\n
                      RX @18:ERR:code=unexpected_type:3E\n
                      RX @2B:STS:rx=7,ae=2,irq=164,pb=164,pm=7,pe=2,qd=0:42\n

--- Resincronizacion: ruido con @ en el medio (caso B del TP) ---
> resync @@@...       TX @008:CMD:ping:52\n
                      RX @0A:ACK:pong=1:22\n

--- Resincronizacion: basura adelante ---
> basura xyz@...      TX xyz@08:CMD:ping:52\n
                      RX @0A:ACK:pong=1:22\n

=== Fin de la prueba ===
```

Figura 3. Verificación del protocolo sobre la placa en el TP5: comandos, estado, errores y resincronización. Esta base se reutilizó sin cambios.

2.4. Periféricos empleados

2.4.1. Conversor analógico-digital (ADC)

El ADC del STM32F103 es de aproximaciones sucesivas, de **12 bits**: convierte una tensión de 0 a 3,3 V en un número de 0 a 4095. Lo usamos para leer el potenciómetro que fija la velocidad. Su reloj no puede superar los 14 MHz, así que lo alimentamos con el reloj de periféricos dividido por 6 (72 MHz / 6 = 12 MHz). El valor crudo se convierte a velocidad porcentual con un mapeo lineal:

$$vel = \frac{ADC \cdot 100}{4095} \quad (2)$$

Para que la lectura no baile con el ruido del cursor del pote, promediamos con un filtro exponencial (EMA) con $\alpha = 1/8$: cada muestra nueva aporta un octavo del valor y el resto lo conserva la historia.

2.4.2. Timers y PWM

Los timers de propósito general cuentan pulsos del reloj de 72 MHz divididos por un prescaler. La frecuencia de desborde (y por lo tanto del PWM) queda definida por dos registros: **PSC** (prescaler) y **ARR** (valor de recarga):

$$f_{PWM} = \frac{f_{clk}}{(PSC+1) \cdot (ARR+1)} \quad (3)$$

En modo PWM el canal compara el contador con un tercer registro (**CCR**): la salida está activa mientras el contador es menor. Variar **CCR** cambia el ciclo de trabajo (duty); variar **ARR** cambia la frecuencia.

2.4.3. Interrupciones externas (EXTI) y UART

El controlador EXTI permite que un flanco en un pin dispare una interrupción sin encuestar el pin en un lazo. Lo usamos para el botón del deadman (ambos flancos: apretar y soltar). La UART (USART1, 115200 baudios, 8N1) es el canal físico del protocolo; a esa velocidad cada byte ocupa 10 bits (arranque + 8 datos + parada):

$$t_{byte} = \frac{10}{115200} \approx 86,8 \mu s \quad (4)$$

de modo que una trama típica de 20 caracteres viaja en menos de 2 ms: el cuello de botella del sistema nunca es la transmisión sino los periodos que elegimos para reportar.

2.5. Sensores de presencia: PIR e inclinación

PIR HC-SR501. Es un sensor piroeléctrico pasivo: detecta cambios en la radiación infrarroja que emite un cuerpo a temperatura humana al moverse frente a su lente de Fresnel. No mide distancia ni temperatura: entrega una salida digital que sube cuando detecta movimiento y se mantiene alta durante un tiempo regulable con un trimmer. Tiene dos particularidades que hay que respetar: necesita un **warm-up** de 30 a 60 segundos al encender, y se alimenta con 5 V aunque su salida es de 3,3 V (compatible directa con el micro).

Sensor de inclinación (tilt). Es el interruptor más simple posible: una bolita conductora dentro de un tubito que cierra o abre el contacto según la orientación. Con el mando derecho la salida queda en un nivel; al inclinarlo o dejarlo caer, cambia. Lo usamos como verificación de **integridad del mando**: un control de hombre muerto tirado en el piso con el botón trabado no debe habilitar al robot.

LA CONDICIÓN DE PRESENCIA

presente = botón apretado $\wedge$ PIR detecta operador $\wedge$ mando derecho. Las tres condiciones se releen cada 20 ms; si cualquiera cae, el sistema pasa a DETENIDO y transmite la parada. Elegimos sensores de salida digital a propósito: no agregan divisores ni canales de ADC y mantienen la placa simple.

3. Diseño del sistema

3.1. Diseño de hardware

3.1.1. Arquitectura general y esquema de conexión

El sistema tiene tres bloques: el **mando** (la Blue Pill con sus sensores y señalización), la **PC del bridge** (que corre el bridge ROS 2 de la cátedra y conversa con la Pill por UART a través de un adaptador USB-serie CH340) y el **robot Unitree**, que solo ve tópicos de ROS 2 y nunca habla directamente con el micro.

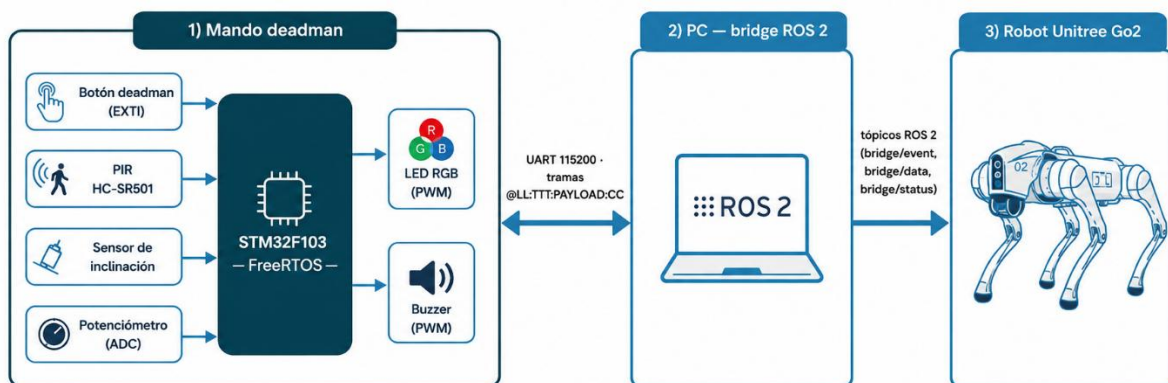


Figura 4. Diagrama en bloques: el mando releva operador y consigna, el bridge traduce tramas a tópicos ROS 2 y el robot ejecuta.

Dentro del mando, cada periférico tiene un pin y un recurso de hardware asignado. El criterio fue agrupar señales por banco (puerto A hacia un lado, puerto B hacia el otro) para simplificar el ruteo de la placa de una sola capa:

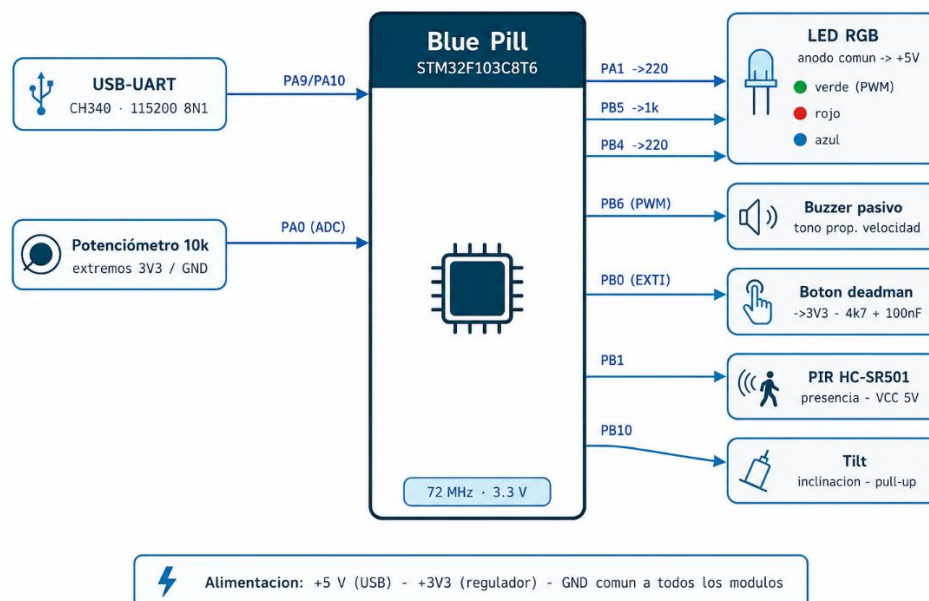


Figura 5. Esquema de conexión: cada periférico con su pin, resistor y alimentación.

Tabla 3. Asignación de pines y recursos del microcontrolador.

Pin	Recurso	Función
PA9 / PA10	USART1	UART al bridge (115200 8N1, cruzada al CH340)
PA0	ADC1_IN0	Cursor del potenciómetro (velocidad)
PA1	TIM2_CH2 (PWM)	LED verde, brillo proporcional a la velocidad
PB5	GPIO	LED rojo (1 kΩ): estado detenido
PB4	GPIO	LED azul (220 Ω): el PIR detecta presencia (pin NJTRST, liberado por firmware)
PB6	TIM4_CH1 (PWM)	Buzzer pasivo, tono proporcional a la velocidad
PB0	EXTI0	Botón deadman (a 3V3, pull-down R3 + filtro de 100 nF)
PB1	GPIO in	Salida del PIR HC-SR501 (módulo alimentado a 5 V)
PB10	GPIO in (pull-up)	Salida del sensor de inclinación

3.1.2. Selección de componentes

Elegimos cada componente con dos criterios: que estuviera disponible en nuestros kits, y que fuera **THT (agujero pasante)** para poder fabricar la placa en casa con pistas anchas.

Tabla 4. Componentes principales y el porqué de cada elección.

Componente	Elección	Por qué
MCU	Blue Pill STM32F103C8T6	La placa de la materia; va socketeada en dos tiras hembra 1×20
LED	RGB 5 mm ánodo común	Tres estados con un solo componente; ánodo a +5 V para que enciendan verde y azul
Buzzer	Pasivo, 12 mm	Necesitamos variar el tono: uno activo suena a frecuencia fija
Botón	Pulsador 12 mm	Cómodo para mantener apretado durante la operación
Velocidad	Potenciómetro 10 kΩ	Consigna analógica directa al ADC, cero componentes extra
Presencia	PIR HC-SR501	Salida digital 3,3 V; detecta operador sin contacto
Integridad	Sensor de inclinación	Digital, trivial de leer; detecta mando caído o inclinado
Serie	CH340 (USB-UART 3,3 V)	El mismo adaptador validado en el TP5

3.1.3. Diseño de la placa de circuito impreso

Dibujamos el esquemático en **KiCad** modelando la Blue Pill como dos conectores de 20 pines (los zócalos reales de la placa) y cada módulo externo (PIR, tilt, RGB, UART) como un header, de modo que todos los sensores quedan enchufables y la Blue Pill se puede retirar. Las conexiones se hicieron con etiquetas de red (PA0, PB6, ...) en lugar de cables largos, lo que mantiene el dibujo legible.

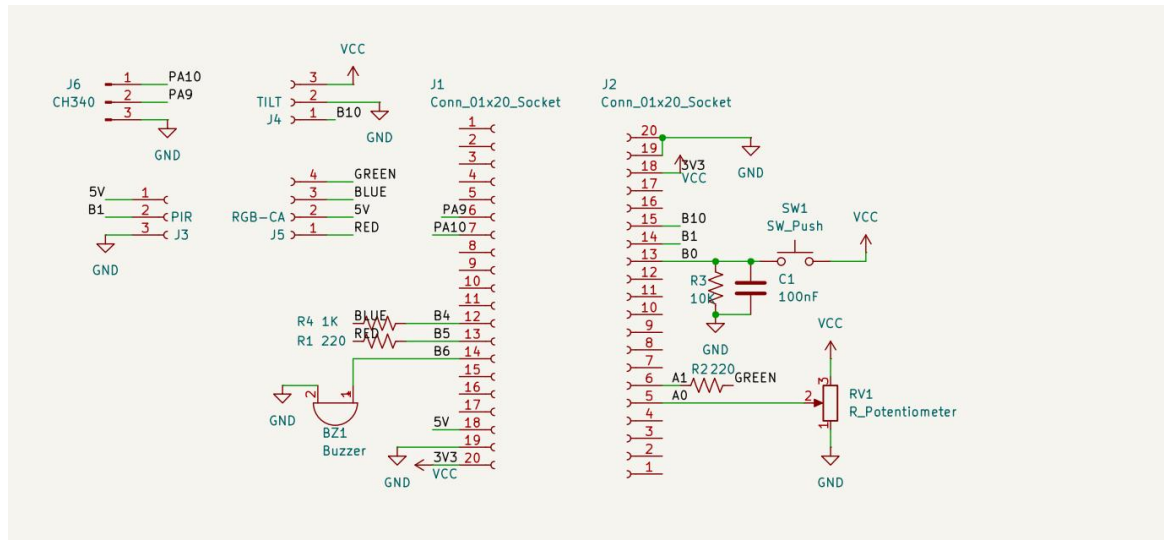


Figura 6. Esquemático del circuito en KiCad: zócalos de la Blue Pill, headers de sensores y UART, RC del botón, LED RGB y buzzer.

Para el ruteo nos impusimos las reglas de la fabricación casera: **una sola capa de cobre** (la inferior), pistas de señal de 0,4 mm como mínimo y alimentación más gruesa, plano de masa por relleno, y los cruces inevitables resueltos con puentes de alambre. El contorno lo dibujamos con forma de **joystick de consola**: además de una estética profesional, las dos “empuñaduras” dejan lugar natural para el pote (zona izquierda) y el pulsador (zona derecha), que es donde van los dedos al operar.

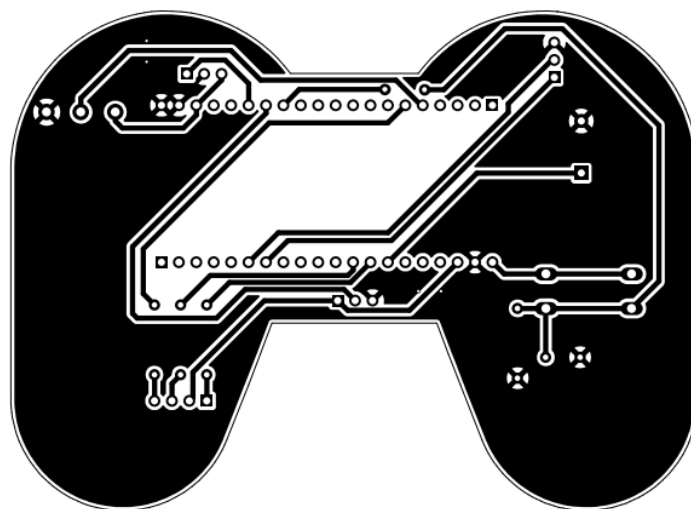


Figura 7. Cara de cobre (una sola capa) lista para transferir: plano de masa, pistas y contorno.

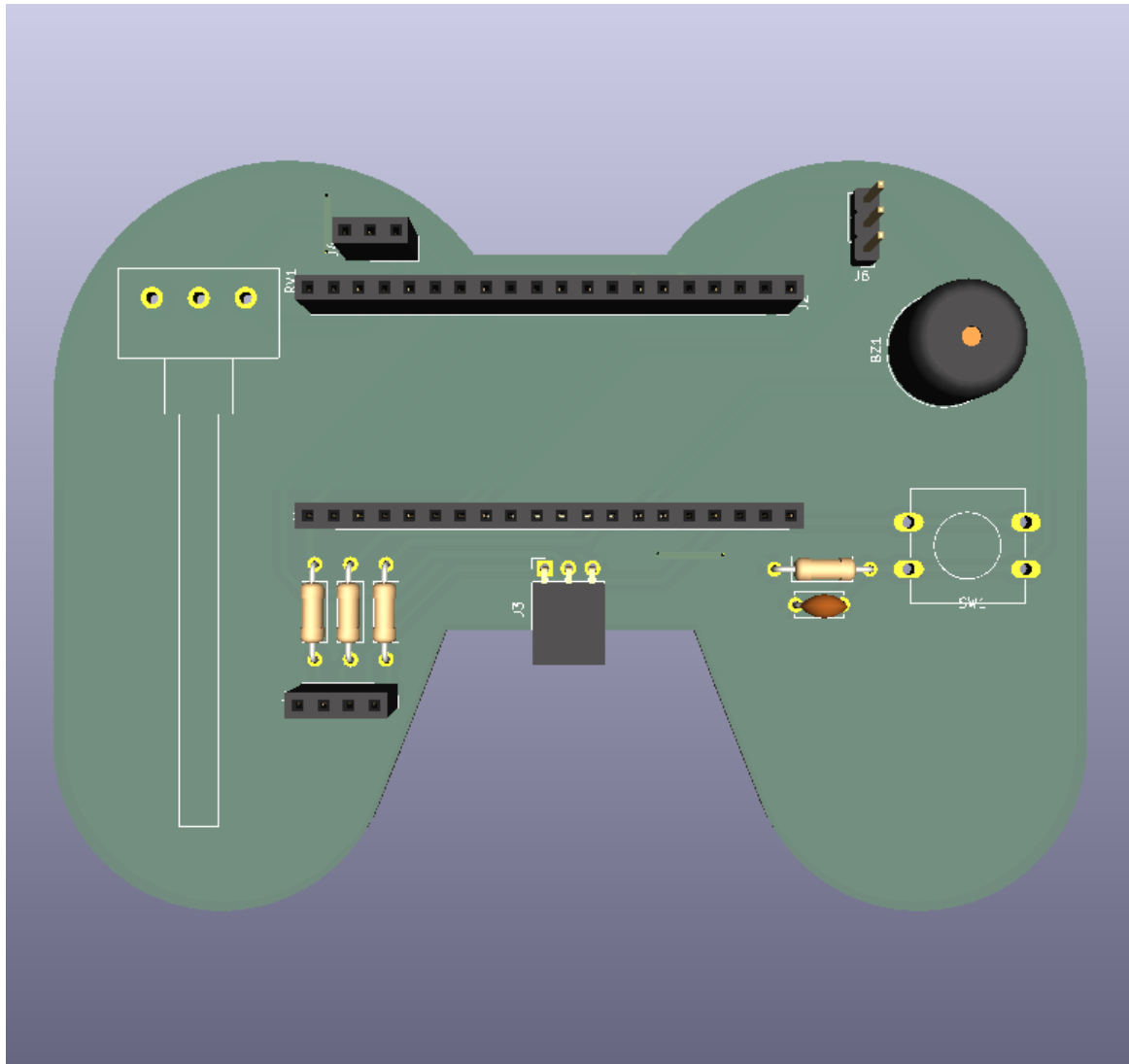


Figura 8. Render 3D del Joystick en KiCad: zócalos de la Blue Pill al centro, pote a la izquierda, botón y buzzer a la derecha, headers para RGB, PIR, tilt y UART.

3.2. Diseño de software

3.2.1. Arquitectura general del firmware

El firmware se organiza en tareas de FreeRTOS conectadas por colas. El camino de recepción es una cadena: la **interrupción de la UART** encola cada byte crudo; `task_parser` los consume de a uno con la FSM del TP5 y, cuando junta una trama válida, la encola como mensaje; `task_app` despacha ese mensaje (en esta variante, el `ACK:ok` del bridge); y `task_uart_tx` toma los mensajes salientes, los codifica con `protocol_encode_frame()` y los transmite. La lógica de la variante vive en su propia tarea:

Tabla 5. Tareas de FreeRTOS del firmware final.

Tarea	Prioridad	Función
<code>task_uart_rx</code>	3	Saca bytes de la cola de la ISR y alimenta al parser
<code>task_parser</code>	3	FSM byte a byte del TP5; entrega mensajes validados

task_app	2	Despacho de mensajes recibidos (ACK del bridge → deadman)
task_deadman	2	La variante: lee sensores, corre la FSM, maneja LED/buzzer y encola tramas
task_actuators	2	LED de placa (PC13) heredado del esqueleto
task_uart_tx	2	Codifica y transmite las tramas salientes

Por interrupción corren cuatro cosas, todas cortas y sin llamadas al RTOS que puedan bloquear: la ISR de la UART (un byte a la cola), **TIM3** a 100 Hz (dispara una conversión del ADC y actualiza el promedio), **EXTI0** (flanco del botón: rearma la ventana de anti-rebote) y **TIM1** en modo one-shot (al vencer los 20 ms de la ventana, fija el estado estable del botón).

3.2.2. Análisis del código: la lógica deadman

El corazón de la variante es una tarea periódica de 20 ms. En cada iteración relee las tres condiciones de presencia y ejecuta las transiciones:

firmware/app/deadman.c (fragmento)

```
bool pressed = button_is_pressed();
bool pir     = presence_pir_active();
bool tilt_ok = presence_tilt_ok();
bool present = pressed && pir && tilt_ok;

if (present && !present_prev) {           /* Ausente -> Presente */
    send_msg(tx, PROTOCOL_TYPE_EVT, "deadman_ok");
} else if (!present && present_prev) {     /* Presente -> Ausente */
    bool sensor_cause = pressed;          /* sigue apretado => fallo PIR/tilt */
    send_msg(tx, PROTOCOL_TYPE_EVT, "deadman");
    if (sensor_cause) {
        send_msg(tx, PROTOCOL_TYPE_STS, "EMERGENCY");
    }
    ack_pending = true;                   /* arranca el reenvio hasta ACK:ok */
}
```

Dos detalles de diseño valen la pena. Primero, la **parada local no espera al bridge**: LED rojo y alarma se activan en el mismo ciclo de 20 ms; el reenvío de **EVT:deadman** cada 500 ms hasta recibir **ACK:ok** es solo para garantizar que la noticia llegue. La seguridad no puede depender de la red. Segundo, distinguimos la causa: si el botón sigue apretado pero cayó un sensor, además del evento se emite **STS:EMERGENCY**, para que del otro lado se sepa que fue la capa de presencia y no la mano del operador.

Los payloads numéricos se generan con ancho fijo y relleno de ceros (`snprintf("spd=%02u")` → `spd=07`), como exige el protocolo. Para el PWM del LED elegimos 1 kHz: bien por encima del parpadeo visible y con 1000 pasos de duty:

firmware/drivers/leds.c (fragmento)

```
timer_set_prescaler(TIM2, 71); /* 72 MHz / 72 = 1 MHz */
timer_set_period(TIM2, 999);   /* 1 MHz / 1000 = 1 kHz */
timer_set_oc_mode(TIM2, TIM_OC2, TIM_OCM_PWM1);
timer_set_oc_polarity_low(TIM2, TIM_OC2); /* anodo comun: bajo = ON */
```


El buzzer usa la misma base de 1 MHz pero con **ARR** variable: para un tono f , $ARR = 10^6/f - 1$ con duty al 50 %. La frecuencia se mapea linealmente de 1 a 4 kHz según la velocidad. Usamos **timers separados** para LED y buzzer a propósito: el LED necesita frecuencia fija y duty variable; el buzzer, frecuencia variable. Como el periodo (ARR) es común a todos los canales de un timer, no se pueden tener las dos cosas en el mismo timer.

El anti-rebote del botón combina EXTI con un timer one-shot: cada flanco reinicia una ventana de 20 ms y, recién cuando la ventana vence sin flancos nuevos, se latched el nivel estable del pin:

firmware/drivers/button.c (fragmento)

```
void exti0_isr(void) { /* cada flanco reinicia la ventana */
    17ven_reset_request(EXTI0);
    timer_set_counter(TIM1, 0U);
    timer_enable_counter(TIM1);
}
void tim1_up_isr(void) { /* venció la ventana: el nivel actual es el estable */
    if (timer_get_flag(TIM1, TIM_SR_UIF)) {
        timer_clear_flag(TIM1, TIM_SR_UIF);
        g_pressed = (gpio_get(GPIOB, GPIO0) != 0U) ? 1U : 0U;
    }
}
```

3.2.3. Diagrama de estados

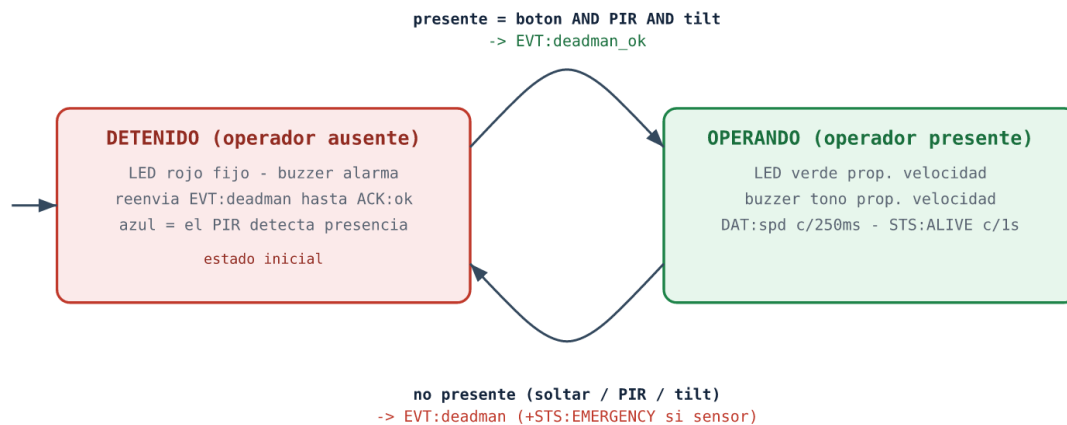


Figura 9. Máquina de estados de la variante: $DETENIDO \rightleftharpoons OPERANDO$, con las tramas de cada transición y los periódicos de cada estado.

Dos refinamientos de interfaz que salieron de usar el mando de verdad: el **LED azul** se apaga durante la operación (para que el brillo del verde se lea limpio como indicador de velocidad) y, en estado detenido, indica que el PIR está detectando presencia —o sea, “estás en rango, podés retomar el control”. Y una **doble pulsación rápida** del botón (dos flancos en menos de 400 ms) alterna el silencio del buzzer, para no tener el tono sonando de continuo durante pruebas largas.

4. Implementación y montaje

4.1. Montaje en protoboard

Antes de fabricar nada validamos todo el sistema en protoboard, con una estrategia incremental que nos dio el propio Makefile: un modo `demo` (Etapa 2 del enunciado) que mapea el pote al brillo y al tono y refleja botón y PIR en los LEDs, sin lógica de variante; un modo `deadman` (el firmware final); y un modo `diag` que reporta por serie el estado crudo de las tres entradas, que terminó siendo la herramienta clave para diagnosticar el sensor de inclinación (sección 8).

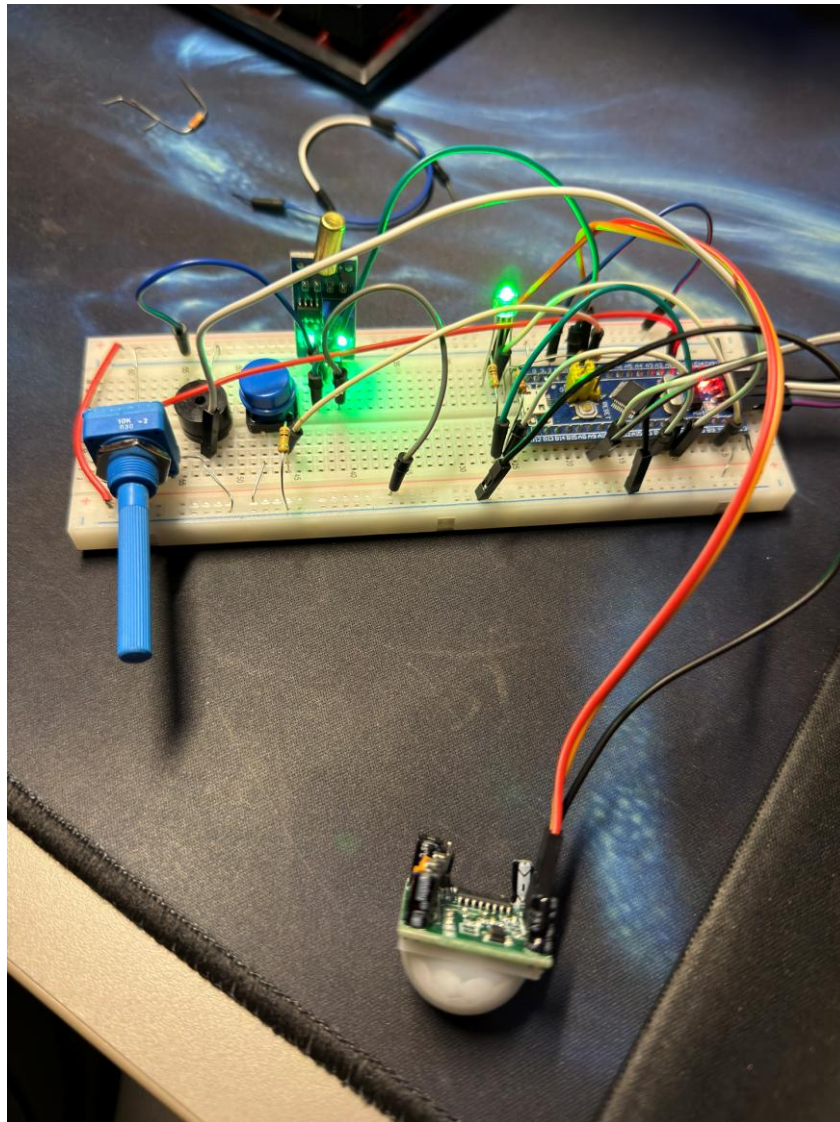


Figura 10. Prototipo completo en protoboard: Blue Pill, pote, botón, buzzer, RGB, tilt y el PIR colgado afuera. Acá se validó todo el firmware antes de hacer la placa.

4.2. Fabricación de la PCB

La placa se fabricó de forma casera con el método de **transferencia de tóner**: imprimimos la cara de cobre espejada en papel ilustración con impresora láser, la transferimos a la placa

virgen con una plancha de ropa, y atacamos el cobre expuesto con **percloruro férrico**. Las reglas de diseño (pistas anchas, aislamiento generoso) están pensadas justamente para que este proceso artesanal salga bien a la primera.

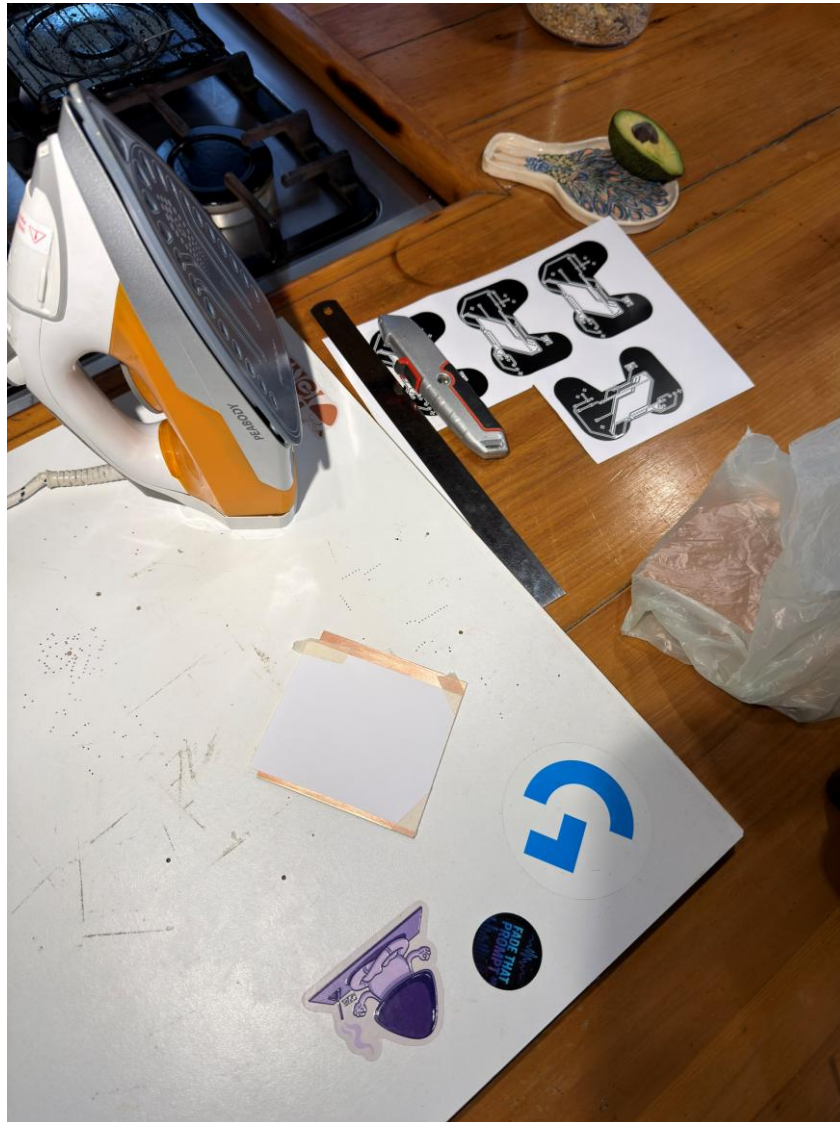


Figura 11. Transferencia de tóner: la impresión espejada de la cara de cobre y la plancha. Imprimimos varias copias del fotolito por si alguna transferencia salía mal.

4.3. Ensamblaje de componentes

Con la placa atacada y perforada soldamos todos los componentes THT. La cara de cobre quedó abajo; el plano de masa rodea las pistas y los pocos cruces se resolvieron con puentes de alambre por la cara de componentes.

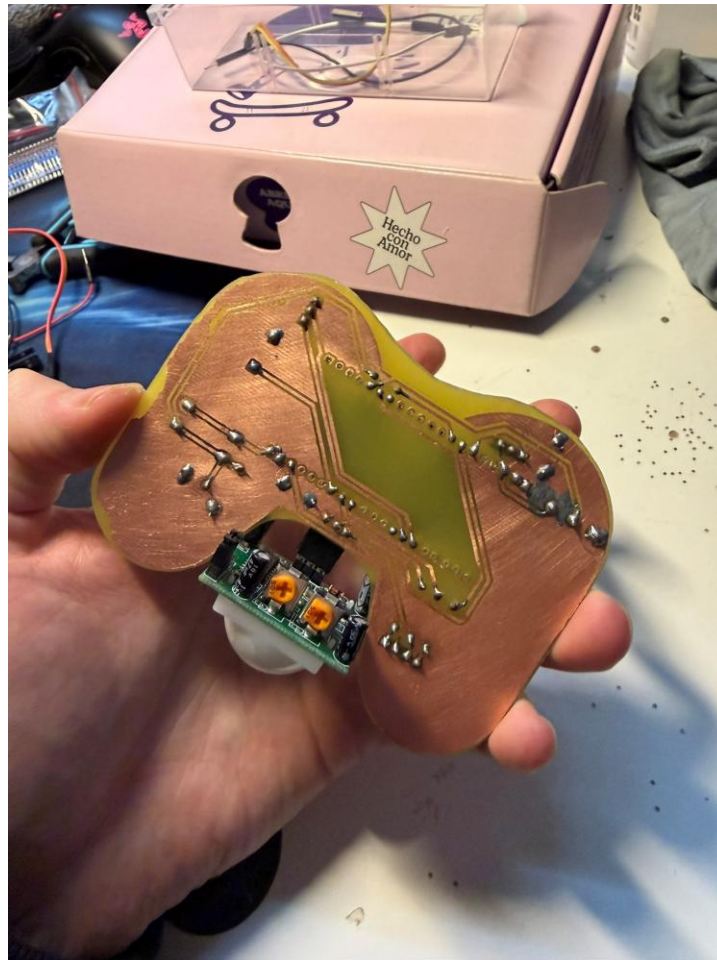


Figura 12. Cara de cobre ya soldada: se aprecia el plano de masa, las pistas y las soldaduras de los zócalos y headers.

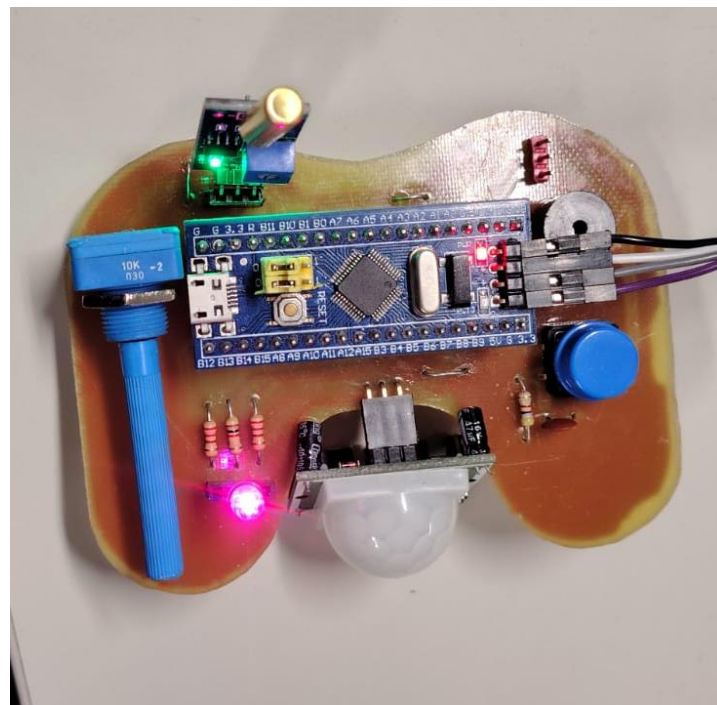


Figura 13. PCB terminada y funcionando: Blue Pill socketeada, PIR y tilt enchufados en sus headers y RGB encendido.

El resultado es un mando compacto donde todo es reemplazable: la Blue Pill sale de sus zócalos, y el PIR, el tilt y el LED RGB se desenchufan de sus headers. Ese criterio nos salvó tiempo en las pruebas y nos deja la puerta abierta a variantes futuras sin re-fabricar.

5. Pruebas y validación

5.1. Configuración del entorno de pruebas

Para poder ensayar el sistema completo sin depender del robot desarrollamos un **emulador del bridge ROS 2 en Python** (`bridge_emulator.py`): abre el puerto serie, valida cada trama recibida (longitud y checksum, igual que el bridge real), la muestra con el **tópico ROS 2** al que se publicaría (`bridge/event`, `bridge/data`, `bridge/status...`), imprime la reacción simulada del robot y **responde ACK:ok a cada EVT:deadman**, cerrando el lazo de reenvío del firmware. La lógica pura del protocolo, además, se re-verificó con los tests de host del TP5: 18 casos del codificador y 20 del parser, todos correctos.

5.2. Ensayos experimentales y resultados

5.2.1. Caso 1: barrido del ADC (consigna de velocidad)

Con el firmware en modo demo giramos el potenciómetro de tope a tope. La telemetría muestra el barrido completo de la escala de 12 bits:

evidencia/etapa2/adc_lecturas.txt (extracto)

```
RX [OK ] DAT:adc=0007    -> bridge/data
RX [OK ] DAT:adc=0276    -> bridge/data
RX [OK ] DAT:adc=0853    -> bridge/data
RX [OK ] DAT:adc=1892    -> bridge/data
RX [OK ] DAT:adc=2922    -> bridge/data
RX [OK ] DAT:adc=3779    -> bridge/data
RX [OK ] DAT:adc=4088    -> bridge/data
```

Los extremos leen 0007 y 4088 en lugar de 0 y 4095: es el efecto esperado del filtro EMA (que nunca llega del todo al valor instantáneo) más la resistencia residual del cursor en los topes. Para la consigna de velocidad es irrelevante: el mapeo a 0–100 % satura correctamente en ambos extremos.

5.2.2. Caso 2: ciclo completo de operación

Con la variante final flasheada verificamos el ciclo entero: tomar el control, variar la velocidad, soltar, y provocar una parada por sensor. La captura muestra la secuencia tal como la ve el bridge:

```

RX [OK ] STS:EMERGENCY    -> bridge/status
    >> ROBOT: emergencia disparada por un sensor
RX [OK ] EVT:deadman_ok   -> bridge/event
    >> ROBOT: operador presente -> habilitado a operar
RX [OK ] DAT:spd=00       -> bridge/data
    >> ROBOT: avanza a 00 % de velocidad
RX [OK ] DAT:spd=00       -> bridge/data
    >> ROBOT: avanza a 00 % de velocidad
RX [OK ] DAT:spd=00       -> bridge/data
    >> ROBOT: avanza a 00 % de velocidad
RX [OK ] EVT:deadman      -> bridge/event
    >> ROBOT: !PARADA INMEDIATA!
TX      ACK:ok            (@06:ACK:ok:4B)
RX [OK ] STS:EMERGENCY    -> bridge/status
    >> ROBOT: emergencia disparada por un sensor
RX [OK ] EVT:deadman_ok   -> bridge/event
    >> ROBOT: operador presente -> habilitado a operar
RX [OK ] DAT:spd=37       -> bridge/data
    >> ROBOT: avanza a 37 % de velocidad
RX [OK ] DAT:spd=37       -> bridge/data
    >> ROBOT: avanza a 37 % de velocidad
RX [OK ] DAT:spd=37       -> bridge/data
    >> ROBOT: avanza a 37 % de velocidad
RX [OK ] EVT:deadman      -> bridge/event
    >> ROBOT: !PARADA INMEDIATA!
TX      ACK:ok            (@06:ACK:ok:4B)
RX [OK ] STS:EMERGENCY    -> bridge/status
    >> ROBOT: emergencia disparada por un sensor
RX [OK ] EVT:deadman_ok   -> bridge/event
    >> ROBOT: operador presente -> habilitado a operar
RX [OK ] DAT:spd=81       -> bridge/data
    >> ROBOT: avanza a 81 % de velocidad
RX [OK ] DAT:spd=81       -> bridge/data
    >> ROBOT: avanza a 81 % de velocidad
RX [OK ] DAT:spd=81       -> bridge/data
    >> ROBOT: avanza a 81 % de velocidad
RX [OK ] EVT:deadman      -> bridge/event
    >> ROBOT: !PARADA INMEDIATA!
TX      ACK:ok            (@06:ACK:ok:4B)
RX [OK ] STS:EMERGENCY    -> bridge/status
    >> ROBOT: emergencia disparada por un sensor
RX [OK ] EVT:deadman_ok   -> bridge/event
    >> ROBOT: operador presente -> habilitado a operar
RX [OK ] DAT:spd=99       -> bridge/data
    >> ROBOT: avanza a 99 % de velocidad

```

Figura 14. Operación real del mando vista por el emulador del bridge: `deadman_ok` al tomar control, `spd=NN` siguiendo al pote (00→37→81→99), paradas con `ACK:ok` y `EMERGENCY` cuando la causa fue un sensor.

Se distinguen las tres cosas que pide la variante: el evento `deadman_ok` al tomar el control (el emulador anota “habilitado a operar”), la telemetría `DAT:spd` cada 250 ms siguiendo fielmente al potenciómetro, y ante cada suelta la secuencia `EVT:deadman` → `ACK:ok` con la parada inmediata.

5.2.3. Caso 3: parada por pérdida de presencia (falla provocada)

El ensayo de falla del enunciado: operando con el botón **apretado**, nos corrimos del campo de visión del PIR. El sistema detectó la ausencia, paró al robot y reportó la causa:

evidencia/etapa4/falla_sensor.txt (extracto)

```

RX [OK ] STS:ALIVE        -> bridge/status

```

```
>> ROBOT: heartbeat de operador presente
RX [OK ] EVT:deadman      -> bridge/event
>> ROBOT: ¡PARADA INMEDIATA!
TX      ACK:ok            (@06:ACK:ok:4B)
RX [OK ] STS:EMERGENCY    -> bridge/status
>> ROBOT: emergencia disparada por un sensor
```

La recuperación es manual a propósito: el sistema no rearma solo, hay que volver a cumplir la condición completa de presencia. Es el comportamiento correcto para un hombre muerto: la máquina nunca debe arrancar sin una decisión explícita del operador. También ensayamos el **timeout de ACK** desconectando el emulador: el firmware reenvía `EVT:deadman` cada 500 ms hasta que el bridge vuelve y confirma.

5.2.4. Demostración en video

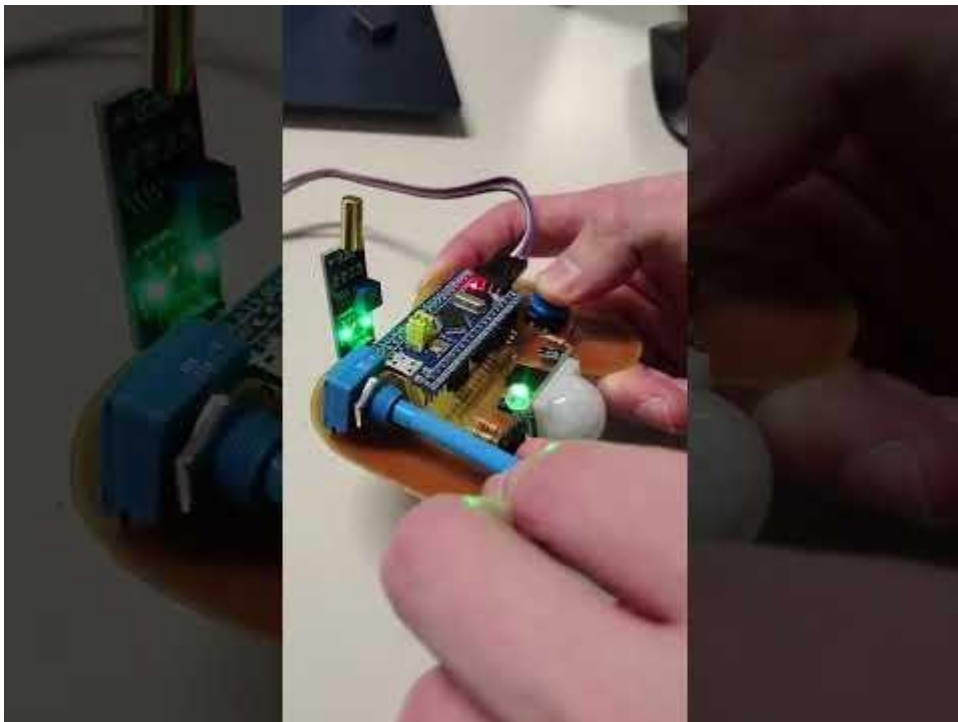


Figura 15. Video de demostración del mando: ciclo completo de operación sobre la PCB.

5.2.5. Integración con el robot

La última etapa es conectar el mando a la computadora del bridge de la cátedra y verificar la reacción del robot físico: avanzar a la velocidad reportada mientras recibe `ALIVE` y detenerse de inmediato al recibir `deadman`.

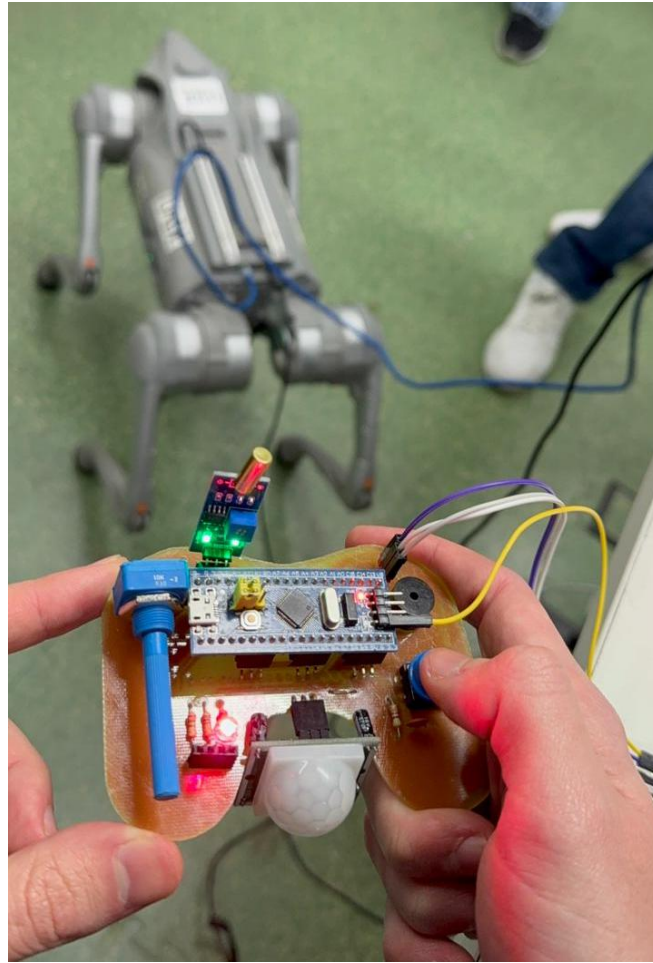


Figura 16. El mando terminado junto al robot cuadrúpedo Unitree durante la integración.

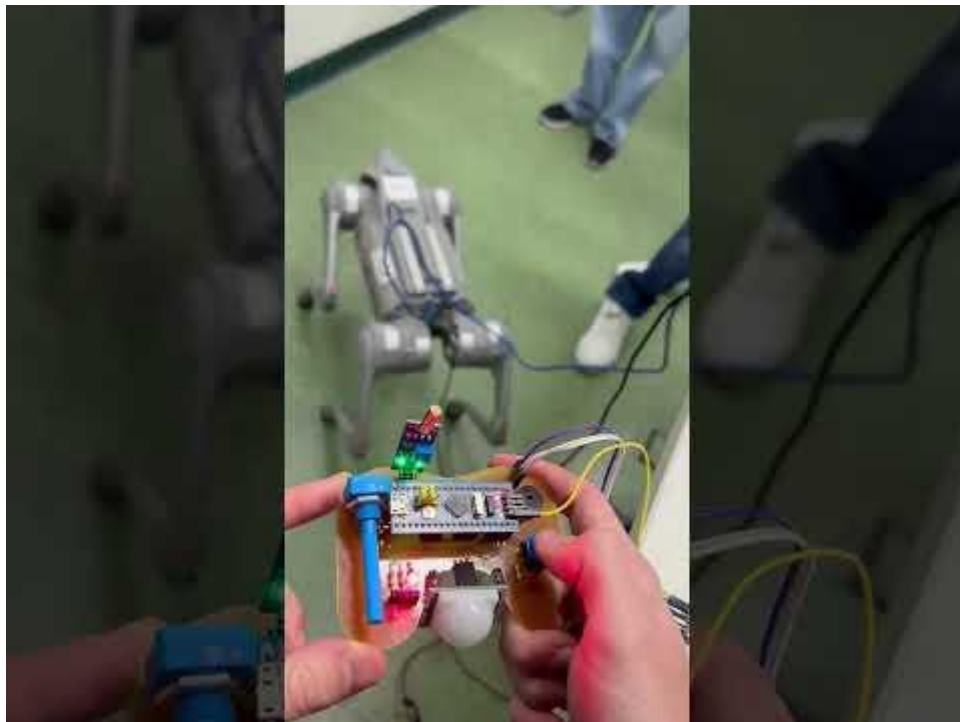


Figura 17. Video de la integración: el robot responde a los eventos del mando a través del bridge ROS 2.

Tabla 6. Resumen de los ensayos y su resultado.

Ensayo	Qué se verificó	Resultado
Tests de host	Codificación, checksum, parser y resincronización (38 casos)	18 + 20 OK
Barrido de ADC	Lectura del pote de tope a tope, telemetría adc=NNNN	0007 → 4088
Ciclo de operación	deadman_ok, spd, ALIVE, parada y ACK	Correcto
Falla por sensor	Parada con botón apretado al perder el PIR; EMERGENCY	Correcto
Timeout de ACK	Reenvío de EVT:deadman cada 500 ms sin bridge	Correcto

6. Especificaciones técnicas

Tabla 7. Especificaciones del sistema terminado.

Parámetro	Valor
Alimentación	5 V por USB (micro-USB de la Blue Pill); lógica interna 3,3 V
Consumo	Del orden de 60–80 mA con PIR y LED activos (estimado, no medido con precisión)
Enlace serie	USART1, 115200 baudios, 8N1, por adaptador CH340 (3,3 V)
Protocolo	Tramas @LL:TTT:PAYLOAD:CC\n con checksum XOR y parser con resincronización
Consigna de velocidad	Potenciómetro 10 k Ω $\rightarrow$ ADC 12 bits $\rightarrow$ 0–100 %, reporte cada 250 ms
Heartbeat	STS:ALIVE cada 1 s mientras el operador está presente
Reintento de parada	EVT:deadman cada 500 ms hasta recibir ACK:ok
Ciclo de la lógica	Tarea deadman cada 20 ms; anti-rebote de 20 ms; ADC a 100 Hz con filtro EMA ($\alpha = 1/8$)
PWM	LED: 1 kHz, duty 0–100 % · Buzzer: 1–4 kHz (tono), duty 50 %
Latencia de parada	Decenas de ms entre el estímulo físico y la trama (ciclo 20 ms + anti-rebote 20 ms + ~ 1 ms de transmisión)
Memoria	≈ 24 KB de flash y $\approx 10,6$ KB de RAM (de 64 KB / 20 KB disponibles)
Placa	PCB propia de una capa, THT, 2 zócalos 1 $\times$ 20 para la Blue Pill, sensores enchufables

7. Procedimientos de calibración y ajustes

El sistema tiene pocos puntos de ajuste y todos están concentrados en lugares previsibles.

7.1. Ajustes de hardware

- ▶ **PIR — sensibilidad y tiempo de retención:** el módulo HC-SR501 trae dos trimmers. El de sensibilidad regula el alcance (horario = más lejos); el de tiempo define cuánto se mantiene alta la salida tras detectar movimiento. Para el uso como deadman conviene retención alta, porque un operador quieto deja de generar detecciones y el sistema lo interpretaría como ausencia.
- ▶ **PIR — jumper de modo:** en H (retrigger) la salida se extiende mientras siga habiendo movimiento; es el modo que usamos.
- ▶ **Tilt — orientación de montaje:** la posición “mando derecho” depende de cómo quede soldado o enchufado el sensor en la placa.

7.2. Ajustes en el código

Si el sensor de inclinación quedara invertido respecto del montaje (nos pasó: sección 8), la corrección es una sola línea en el driver:

`firmware/drivers/presence.c`

```
bool presence_tilt_ok(void)
{
    return gpio_get(GPIOB, GPIO10) == 0U; /* reposo = bajo = estable */
}
```

El resto de los parámetros operativos vive en un único archivo de configuración, pensado para tocar sin bucear en los drivers:

Tabla 8. Constantes de ajuste en `config/app_config.h`.

Constante	Valor	Qué controla
DEADMAN_TASK_PERIOD_MS	20	Periodo del lazo de la variante
DEADMAN_SPD_PERIOD_MS	250	Cadencia del reporte de velocidad
DEADMAN_ALIVE_PERIOD_MS	1000	Cadencia del heartbeat
DEADMAN_ACK_TIMEOUT_MS	500	Reintento del evento de parada
DEADMAN_DOUBLE_TAP_MS	400	Ventana de la doble pulsación (silenciar buzzer)
BUZZER_FREQ_MIN/MAX_HZ	1000 / 4000	Rango de tonos según velocidad
BUTTON_DEBOUNCE_TIM_*	20 ms	Ventana del anti-rebote por hardware

8. Registro anecdótico y seguimiento de fallas

Esta sección documenta, en orden cronológico, los problemas reales que tuvimos y cómo los diagnosticamos. Fue la parte del trabajo donde más aprendimos: ninguna de estas fallas era de la lógica del programa —todas estaban en la frontera entre el firmware y el mundo físico.

8.1. El LED RGB que casi no encendía (y el azul que no encendía nunca)

Primer síntoma: los tres colores del RGB brillaban poquísimo, y el azul directamente nada. Primera sospecha (el LED quemado) descartada probándolo afuera. La causa era eléctrica: habíamos llevado el **ánodo común a 3,3 V**, pero el verde y el azul tienen una tensión directa de 3,0–3,4 V: a 3,3 V casi no queda margen para hacer circular corriente. Movimos el ánodo común a **+5 V** y los colores aparecieron. Es seguro para el micro: cuando el pin está en alto (color apagado), la propia caída del LED mantiene el nodo por debajo de 3,3 V.

EFFECTO SECUNDARIO Y SU SOLUCIÓN

Con 5 V en el ánodo, el rojo (que cae solo ~2 V) quedaba apenas encendido aun “apagado”, porque el pin en alto llega a 3,3 V y no a 5 V: quedaban ~1,7 V residuales, suficientes para un hilito de luz roja. Subimos su resistor de 220 Ω a 1 k Ω y el filtrado se volvió invisible (el verde y el azul no filtran: su tensión directa es mayor que el residuo).

8.2. El pin PB4 que no era un GPIO

Aun con la alimentación corregida, el azul seguía muerto —incluso puentando su señal directamente a 3,3 V. La explicación está en el manual de referencia: **PB4 es NJTRST**, un pin de la interfaz JTAG, y por defecto *no funciona como GPIO*. Hay que liberarlo por software remapeando la interfaz de depuración (manteniendo SWD, que es por donde flasheamos):

```
firmware/platform/system_init.c
rcc_periph_clock_enable(RCC_AFIO);
/* Libera NJTRST (PB4) como GPIO manteniendo SWD activo */
gpio_primary_remap(AFIO_MAPR_SWJ_CFG_FULL_SWJ_NO_JNTRST, 0);
```

Una línea de código, una tarde de diagnóstico. Clásico del STM32F103.

8.3. El botón fantasma

En las pruebas del PIR notamos algo raro: al mover la mano cerca del protoboard, a veces “se apretaba” el botón solo (sonaba el buzzer un instante). El pin del botón, en alta impedancia y con cables largos, estaba **tomando el acople capacitivo de la mano**. Lo resolvimos endureciendo la entrada: pull-down de **4,7 k Ω** bien asentado más un capacitor de **100 nF** del pin a masa (un RC de ~0,5 ms que se come los glitches). Desapareció por completo, y el filtro

quedó incorporado al diseño de la PCB (dimensionado como $R3 = 10\text{ k}\Omega + 100\text{ nF}$, que da una constante de tiempo similar con más margen).

8.4. El tilt que bloqueaba todo

Con la variante flasheada, el sistema no pasaba nunca a OPERANDO aunque el botón y el PIR estuvieran bien. Para no adivinar, compilamos el **modo diagnóstico** (`make DIAG=1`), que reporta el estado crudo de las tres entradas cada 300 ms. El culpable apareció al instante: `b=1,p=1,t=0`. Nuestro módulo de tilt deja la salida en **bajo** con el mando derecho —al revés de lo que habíamos supuesto. Un desconectar-el-pin de confirmación (con pull-up, pin suelto lee alto → el verde encendió) y la inversión de una línea en `presence.c` cerraron el tema.

8.5. El Makefile que flasheaba el firmware equivocado

La falla más traicionera no estaba en el hardware ni en el firmware, sino en el **proceso de build**. Usábamos `make MODE=demo && make flash` y la placa seguía corriendo la versión anterior. Dos causas encadenadas: primero, `make` decide recompilar por fecha de archivos y no mira los flags `-D`, así que cambiar el modo no recompilaba nada; y segundo, el `make flash` del final —sin `MODE`— volvía a compilar con el modo por defecto y flasheaba eso. Lo resolvimos con un **sello de modo** (un archivo vacío cuyo nombre codifica el modo compilado, del que dependen los objetos) y targets de un solo paso: `make flash-demo`, `make flash-deadman`, `make flash-diag`.

LA MORALEJA DEL REGISTRO

Todas las fallas se resolvieron igual: aislar la variable antes de tocar nada (probar el LED afuera, puentear un pin, desconectar un sensor, mirar el símbolo dentro del binario con `nm`). El modo diagnóstico que agregamos al firmware nació de esta necesidad y quedó como herramienta permanente del proyecto.

9. Parte comercial

9.1. Costos y presupuesto

Valores aproximados en pesos, a junio de 2026, de los componentes del mando. La mayoría salió de kits que ya teníamos.

Tabla 9. Presupuesto aproximado del mando (valores de referencia, jun-2026).

Ítem	Costo aprox. (ARS)
Blue Pill STM32F103C8T6	9.000
Adaptador USB-serie CH340	4.000
Módulo PIR HC-SR501	3.500
Módulo sensor de inclinación	1.500
Potenciómetro, pulsador, buzzer pasivo, LED RGB	3.500
Resistores, capacitores, headers y zócalos	3.000
Placa virgen FR-4 + percloruro + insumos de transferencia	6.000
Total	≈ 30.500

Como referencia de escala: el mando completo cuesta menos que una hora de uso de un robot como el Go2. Tiene sentido que la capa de seguridad física sea barata, reemplazable y hecha a medida.

10. Conclusiones

- ▶ El sistema completo funciona de punta a punta: **38 casos de test de host en verde**, el ciclo de operación validado sobre la placa real y la parada por pérdida de presencia respondiendo en decenas de milisegundos.
- ▶ La decisión de reutilizar el protocolo del TP5 **sin modificarlo** pagó: no tocamos ni una línea del parser en todo el integrador, y toda la energía fue a la variante y al hardware.
- ▶ La capa de presencia (PIR + tilt) convierte el deadman de manual en algo más cercano a un sistema de seguridad real: el mando distingue “operador que suelta” de “operador que desaparece” y lo reporta distinto (**STS:EMERGENCY**).
- ▶ La arquitectura de tareas y colas de FreeRTOS demostró su valor en la práctica: agregar la tarea de la variante no requirió tocar la cadena de recepción ni la de transmisión.
- ▶ Fabricar la PCB nos dejó el aprendizaje más transferible: las reglas de diseño (una capa, pistas anchas, componentes socketeados) no son estética, son lo que hace viable el proceso casero con percloruro.
- ▶ El firmware ocupa ≈ 24 KB de flash y $\approx 10,6$ KB de RAM: queda holgura de sobra para crecer (un 62 % de flash libre).

11. Posibilidades de mejora

- ▶ **CRC-8 en lugar de XOR:** misma huella en la trama (dos dígitos hex), mucha más cobertura de errores, incluidos intercambios de bytes y ráfagas.
- ▶ **Transmisión UART por interrupción o DMA:** hoy la transmisión es bloqueante byte a byte; con DMA la tarea de TX quedaría libre al instante.
- ▶ **Watchdog independiente (IWDG):** si el firmware se colgara, el robot seguiría recibiendo el último estado; un watchdog garantiza reinicio y silencio de heartbeat (que el bridge interpreta como parada).
- ▶ **Batería y enlace inalámbrico:** el mando hoy es cautivo del cable USB-serie; con una celda LiPo y un transceptor el deadman sería portátil de verdad.
- ▶ **Gabinete impreso en 3D:** la forma de gamepad de la placa está pidiendo una carcasa a juego, con el PIR mirando al operador.
- ▶ **Medición formal de latencia:** instrumentar con un pin de test y osciloscopio el tiempo botón→trama para reemplazar la estimación por un número medido.

12. Manual de usuario

12.1. Normas de seguridad, precauciones y recomendaciones

- ▶ El mando agrega una capa de monitoreo: **no reemplaza** la parada de emergencia nativa ni los sensores del robot. No puentear ni desactivar los sistemas de seguridad del fabricante.
- ▶ La Blue Pill trabaja a 3,3 V: no conectar señales de 5 V a ningún pin. El PIR se alimenta de 5 V pero su salida es de 3,3 V (así está cableado en la placa).
- ▶ Mantener masa común entre el mando, el adaptador serie y la PC del bridge. El conexionado hacia el robot lo realiza la cátedra.
- ▶ Ante cualquier comportamiento extraño del robot, soltar el botón: la parada es la acción por defecto y siempre está a un dedo de distancia.

12.2. Instrucciones de uso

Puesta en marcha

- ▶ Conectar el adaptador USB-serie a la PC del bridge y alimentar el mando por su micro-USB.
- ▶ Esperar el warm-up del PIR (30–60 s). El LED azul encendido indica que el sensor ya detecta al operador.

Operación

- ▶ Pararse frente al mando, sostenerlo derecho y **mantener presionado el botón**. El LED pasa a verde: el robot queda habilitado.
- ▶ Regular la velocidad con la perilla: el brillo del verde y el tono del buzzer acompañan la consigna.
- ▶ Para detener: soltar el botón (o, automáticamente, si el mando se inclina o el operador sale del campo del PIR). El LED pasa a rojo fijo y suena la alarma intermitente.
- ▶ Doble pulsación rápida del botón: silencia o reactiva el buzzer (útil en sesiones largas de prueba).

Tabla 10. Código de señalización del mando.

Señal	Significado
Verde (brillo variable)	Operando; el brillo refleja la velocidad comandada
Rojo fijo	Detenido (operador ausente o mando inclinado)
Azul (solo detenido)	El PIR detecta presencia: se puede retomar el control
Tono variable	Operando; la frecuencia sube con la velocidad

Alarma intermitente	Parada activa; cesa al retomar el control (o con doble pulsación)
---------------------	---

13. Bibliografía y recursos

- ▶ Apuntes de la cátedra de Sistemas Embebidos (UADE, 2026): periféricos del STM32F103, protocolo serie ROS 2 / Blue Pill, máquinas de estados, FreeRTOS.
- ▶ Enunciados del TP5 (protocolo serie) y del TP Integrador Blue Pill y Unitree — Dr. Ing. Leonardo J. Amet.
- ▶ ST Microelectronics — RM0008 Reference Manual (STM32F101/102/103/105/107) y datasheet del STM32F103C8T6.
- ▶ Documentación de libopencm3 (biblioteca de periféricos) y de FreeRTOS (kernel de tiempo real).
- ▶ Hoja de datos del sensor PIR HC-SR501.
- ▶ Documentación de KiCad E.D.A. (esquemático y PCB).

14. Anexos

Anexo A: repositorio del proyecto

El código fuente completo (firmware, tests de host, emulador del bridge), la evidencia por etapas y este informe están en el repositorio del grupo:

[\[REPOSITORIO DE GITHUB – TP-Integrador-Embebidos \]](#)

Estructura del repositorio:

codigo/firmware/	firmware (drivers, app/deadman.c, protocolo del TP5)
codigo/host_tests/	tests de host del protocolo y el parser (make run)
codigo/serial_test/	bridge_emulator.py (emulador del bridge ROS 2)
evidencia/	capturas y registros por etapa
informe/	este documento

Anexo B: compilación y flasheo

El firmware se compila con make (toolchain arm-none-eabi + OpenOCD). Los targets compilan y flashean el modo correcto en un solo paso:

```
make flash-deadman # variante deadman (firmware final)
make flash-demo    # demo de perifericos (Etapa 2 del enunciado)
make flash-diag    # variante + diagnostico de entradas por serie
make size          # uso de memoria
cd codigo/host_tests && make run # tests de host (38 casos)
```